

Práctico 5 - Jerarquía de Memorias

AdC - 2025

Fórmulas útiles:

$$\# \text{ Tags} = \frac{\# \text{ bloques de memoria}}{\# \text{ líneas de la caché}}$$

$$\# \text{ bloques de memoria} = \frac{\# \text{ Palabras de la memoria}}{\# \text{ Palabras en un bloque de caché}}$$

$$\# \text{ Líneas de la caché} = \frac{\text{Tamaño sección datos de la caché}}{\text{Tamaño de un bloque / línea}} .$$

$$AMAT = \overset{(\text{access time})}{\text{Hit-Time}} + \text{miss rate} * \text{penalty}$$

$$CPI_{avg} = CPI_{base} + \left(\frac{\text{Memory Accesses}}{\text{Total instructions}} \right) * \overset{\text{miss}}{\text{rate}} * \text{penalty}$$

Ejercicio 1:

Indicar si son correctas o no las siguientes afirmaciones, si se intenta realizar una comparación entre distintos sistemas de memoria tecnológicamente diferentes. Justificar su respuesta.

- a. A menor tiempo de acceso, mayor coste por bit. **V**
- b. A menor capacidad, menor coste por bit. **F**
- c. A mayor capacidad, menor tiempo de acceso. **F**
- d. A mayor tiempo de acceso, mayor capacidad. **V**
- e. A mayor frecuencia de acceso, menor capacidad y mayor coste por bit. **V**

Las memorias con mayor coste por bit son aquellas con menor tiempo de acceso y menor capacidad. SRAM (Caché) >> DRAM (RAM) >> Disco

Ejercicio 2:

Considerando un procesador que trabaja a 1.7GHz y los siguientes tiempos de acceso a memoria:

Memory technology	Typical access time	\$ per GiB in 2012
SRAM semiconductor memory	0.5–2.5ns	\$500–\$1000
DRAM semiconductor memory	50–70ns	\$10–\$20
Flash semiconductor memory	5,000–50,000 ns	\$0.75–\$1.00
Magnetic disk	5,000,000–20,000,000ns	\$0.05–\$0.10

¿Cuántos ciclos de clock implica leer un dato de caché (SRAM) y de memoria principal (DRAM)?

$$1,7 \text{ GHz} = \frac{1}{1,7 \times 10^9} \text{ s} = \underline{0,58 \text{ ns un ciclo de clock}}$$

$$\Rightarrow \text{SRAM (0,5 ns de access time)} = \frac{0,5}{0,58} = 0,86 \text{ ciclo de clock}$$

$$\Rightarrow \text{DRAM (50 ns de access time)} = \frac{50}{0,58} = 86,2 \text{ ciclos de clock}$$

Ejercicio 3:

Calcular el tamaño total (en bits) de una caché de mapeo directo de 16KiB (16K x 8 bits) de datos y tamaño de bloque de 4 palabras de 32 bits c/u. Asuma direcciones de 64 bits.

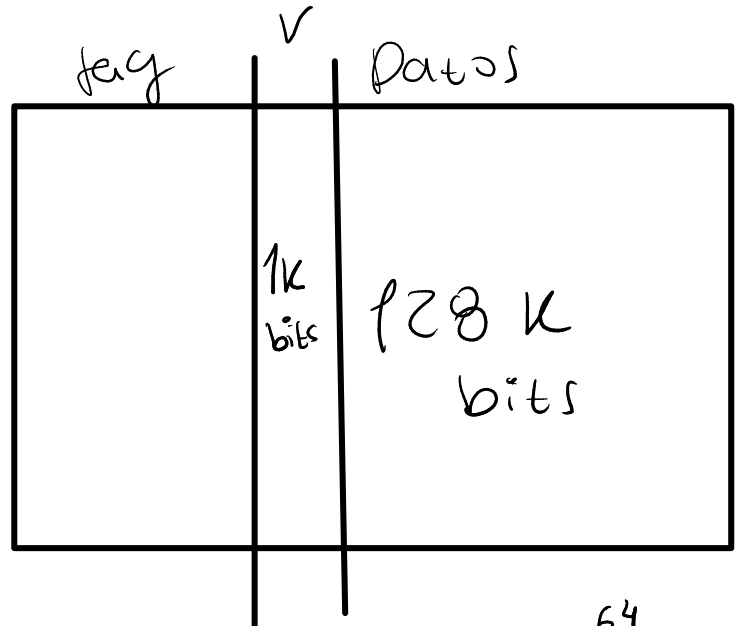
$$\text{Datos (cache)}: 16 \text{ KiB } (16k \times 8 \text{ bits}) = 2^4 \times 2^{10} \times 2^3 = 2^{17} \text{ bits} = 128k \text{ bits}$$

$$\text{Bloque} : 4 \times 32 = 128 \text{ bits}$$

$$\text{Tamaño de la ram} = 2^{64} \text{ bits}$$

$$\text{Líneas de la caché} = \frac{\text{tamaño total de los datos}}{\text{tamaño de un bloque}}$$

$$= \frac{128k}{128} = 1k \text{ líneas}$$



$$\text{Bits de tag} = \frac{\text{Cant. bloques mem}}{\text{Líneas de caché}}$$

W_{mem} = palabra de la memoria

W_p = palabra del proce
(formada por varios W_{mem})

$$= \frac{2^{60}}{2^{10}} = 2^{50}$$

tags distintos en la caché

$$\frac{\text{cant } W_{mem}}{\text{cant } W_{mem} \text{ en } W_p} = \frac{2^{64}}{2^4}$$

$$4 \text{ de } 32 \text{ (8+8+8+8)} = 16 W_{mem} \text{ en } W_p$$

$$\text{Bits de tag} = 50$$

Entonces: Datos: 128k b

Bit de verificación: 1k b

Tags: 50k b

Tamaño total: 179k b

Ejercicio 4:

Las memorias caché son fundamentales para elevar el rendimiento de un sistema de memoria jerárquico respecto del procesador. A continuación se da una lista de referencias de acceso a memoria (direcciones de 64 bits) las cuales deben ser consideradas como accesos secuenciales en ese mismo orden. El formato que se utiliza para cada dirección está reducido a sólo 16 bits, solo con fines prácticos:

0x000C, 0x02D0, 0x00AC, 0x0008, 0x02FC, 0x0160,
0x02F8, 0x0038, 0x02D4, 0x00AC, 0x00B0, 0x0160

Se debe:

- Para cada una de estas referencias a memoria, determinar el binario de la dirección de cada palabra (cada palabra de 32 bits), la etiqueta (tag), el número de línea (index) asignado en una cache de mapeo directo, con un tamaño de 16 bloques de 1 palabra c/u. Además liste qué referencias produjeron un acierto (hit) o un fallo (miss) de caché, suponiendo que la cache se inicializa vacía.

Líneas cache = 16, tamaño = 32 bits (4 bytes = 4 wmem = 1 wq)

Bits de index = 4, bits de offset = 2

$$\text{Cant. tags en la cache} = \frac{\text{Bloques de mem}}{\text{Bloques de cache}} = \frac{\frac{\text{Cant wmem}}{\text{tamaño wq}}}{\frac{\text{bloques cache}}{2^4}} = \frac{2^{64}}{2^2} = 2^{58}$$

$\Rightarrow$ bits de tag = 58

Formato de dirección =

tag	index	w	off
58	4	0	2

	tag	v	datos	
0		0		
1		0		
2	0x0	0	mem[0x08 a 0x0B]	Miss
3	0x0	1	mem[0xC a 0xF]	Miss
4	0x0B	1	mem[0x02D0 a 0x02D3]	Miss
5		0		
6		0		
7		0		
8		0		
9		0		
A		0		
B	0x02	1	mem[0xAC a 0xAF]	Miss
C		0		
D		0		
E		0		
F	0x0B	0	mem[0x02FC a 0x02FF]	Miss

1. 0x000C: 00001100

2. 0x02D0: 001011010000

3. 0x00AC: 0000101100

4. 0x0008: 00001000

5. 0x02FC: 0010111100

6. 0x0160: 0001011000

7. 0x02F8: 0010111100

8. 0x0038: 0000111000

9. 0x02D4: 0010110100

10. 0x00AC: 0000101100

11. 0x00B0: 0000101100

12. 0x0160: 0001011000

13. 0x02FC: 0010111100

14. 0x02FF: 0010111100

15. 0x02FF: 0010111100

16. 0x02FF: 0010111100

17. 0x02FF: 0010111100

18. 0x02FF: 0010111100

19. 0x02FF: 0010111100

20. 0x02FF: 0010111100

21. 0x02FF: 0010111100

22. 0x02FF: 0010111100

23. 0x02FF: 0010111100

24. 0x02FF: 0010111100

25. 0x02FF: 0010111100

26. 0x02FF: 0010111100

27. 0x02FF: 0010111100

28. 0x02FF: 0010111100

29. 0x02FF: 0010111100

30. 0x02FF: 0010111100

31. 0x02FF: 0010111100

32. 0x02FF: 0010111100

33. 0x02FF: 0010111100

34. 0x02FF: 0010111100

35. 0x02FF: 0010111100

36. 0x02FF: 0010111100

37. 0x02FF: 0010111100

38. 0x02FF: 0010111100

39. 0x02FF: 0010111100

40. 0x02FF: 0010111100

41. 0x02FF: 0010111100

42. 0x02FF: 0010111100

43. 0x02FF: 0010111100

44. 0x02FF: 0010111100

45. 0x02FF: 0010111100

46. 0x02FF: 0010111100

47. 0x02FF: 0010111100

48. 0x02FF: 0010111100

49. 0x02FF: 0010111100

50. 0x02FF: 0010111100

51. 0x02FF: 0010111100

52. 0x02FF: 0010111100

53. 0x02FF: 0010111100

54. 0x02FF: 0010111100

55. 0x02FF: 0010111100

56. 0x02FF: 0010111100

57. 0x02FF: 0010111100

58. 0x02FF: 0010111100

59. 0x02FF: 0010111100

60. 0x02FF: 0010111100

61. 0x02FF: 0010111100

62. 0x02FF: 0010111100

63. 0x02FF: 0010111100

64. 0x02FF: 0010111100

65. 0x02FF: 0010111100

66. 0x02FF: 0010111100

67. 0x02FF: 0010111100

68. 0x02FF: 0010111100

69. 0x02FF: 0010111100

70. 0x02FF: 0010111100

71. 0x02FF: 0010111100

72. 0x02FF: 0010111100

73. 0x02FF: 0010111100

74. 0x02FF: 0010111100

75. 0x02FF: 0010111100

76. 0x02FF: 0010111100

77. 0x02FF: 0010111100

78. 0x02FF: 0010111100

79. 0x02FF: 0010111100

80. 0x02FF: 0010111100

81. 0x02FF: 0010111100

82. 0x02FF: 0010111100

83. 0x02FF: 0010111100

84. 0x02FF: 0010111100

85. 0x02FF: 0010111100

86. 0x02FF: 0010111100

87. 0x02FF: 0010111100

88. 0x02FF: 0010111100

89. 0x02FF: 0010111100

90. 0x02FF: 0010111100

91. 0x02FF: 0010111100

92. 0x02FF: 0010111100

93. 0x02FF: 0010111100

94. 0x02FF: 0010111100

95. 0x02FF: 0010111100

96. 0x02FF: 0010111100

97. 0x02FF: 0010111100

98. 0x02FF: 0010111100

99. 0x02FF: 0010111100

100. 0x02FF: 0010111100

101. 0x02FF: 0010111100

102. 0x02FF: 0010111100

103. 0x02FF: 0010111100

104. 0x02FF: 0010111100

105. 0x02FF: 0010111100

106. 0x02FF: 0010111100

107. 0x02FF: 0010111100

108. 0x02FF: 0010111100

109. 0x02FF: 0010111100

110. 0x02FF: 0010111100

111. 0x02FF: 0010111100

112. 0x02FF: 0010111100

113. 0x02FF: 0010111100

114. 0x02FF: 0010111100

115. 0x02FF: 0010111100

116. 0x02FF: 0010111100

117. 0x02FF: 0010111100

118. 0x02FF: 0010111100

119. 0x02FF: 0010111100

120. 0x02FF: 0010111100

121. 0x02FF: 0010111100

122. 0x02FF: 0010111100

123. 0x02FF: 0010111100

124. 0x02FF: 0010111100

125. 0x02FF: 0010111100

126. 0x02FF: 0010111100

127. 0x02FF: 0010111100

128. 0x02FF: 0010111100

129. 0x02FF: 0010111100

130. 0x02FF: 0010111100

131. 0x02FF: 0010111100

132. 0x02FF: 0010111100

133. 0x02FF: 0010111100

134. 0x02FF: 0010111100

135. 0x02FF: 0010111100

136. 0x02FF: 0010111100

137. 0x02FF: 0010111100

138. 0x02FF: 0010111100

139. 0x02FF: 0010111100

140. 0x02FF: 0010111100

141. 0x02FF: 0010111100

142. 0x02FF: 0010111100

143. 0x02FF: 0010111100

144. 0x02FF: 0010111100

145. 0x02FF: 0010111100

146. 0x02FF: 0010111100

147. 0x02FF: 0010111100

148. 0x02FF: 0010111100

149. 0x02FF: 0010111100

150. 0x02FF: 0010111100

151. 0x02FF: 0010111100

152. 0x02FF: 0010111100

153. 0x02FF: 0010111100

154. 0x02FF: 0010111100

155. 0x02FF: 0010111100

156. 0x02FF: 0010111100

157. 0x02FF: 0010111100

158. 0x02FF: 0010111100

159. 0x02FF: 0010111100

160. 0x02FF: 0010111100

161. 0x02FF: 0010111100

162. 0x02FF: 0010111100

163. 0x02FF: 0010111100

164. 0x02FF: 0010111100

165. 0x02FF: 0010111100

166. 0x02FF: 0010111100

167. 0x02FF: 0010111100

168. 0x02FF: 0010111100

169. 0x02FF: 0010111100

170. 0x02FF: 0010111100

171. 0x02FF: 0010111100

172. 0x02FF: 0010111100

173. 0x02FF: 0010111100

174. 0x02FF: 0010111100

175. 0x02FF: 0010111100

176. 0x02FF: 0010111100

177. 0x02FF: 0010111100

178. 0x02FF: 0010111100

179. 0x02FF: 0010111100

180. 0x02FF: 0010111100

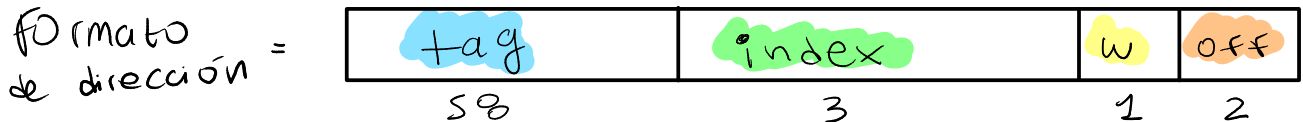
181. 0x02FF: 0010111100

182. 0x02FF: 0010111100

183. 0x02FF: 00101111

- b. Para cada una de estas referencias a memoria, determinar el binario de la dirección de cada palabra (cada palabra de 32 bits), la etiqueta (tag), el número de línea (index) asignado en una cache de mapeo directo, con un tamaño de 8 bloques de 2 palabras c/u. Además liste qué referencias produjeron un acierto (hit) o un fallo (miss) de caché, suponiendo que la cache se inicializa vacía.

Se producirá un **HIT** cuando se acceden a direcciones contiguas de palabras guardadas en la cache.



datos					
	tag	v	w ₀	w ₁	
0		0			
1	0x0	1	0x08 - 0x0B	0x0C - 0x0F	M; H
2	0xB	7	0x200 - 0x203	0x204 - 0x207	M
3		0			
4	0x05	1	0x150 - 0x153	0x154 - 0x157	M; H
5	0x02	1	0x0A8 - 0x0AB	0x0AC - 0x0AF	M; H
6	0x02	1	0x0B0 - 0x0B3	0x0B4 - 0x0B7	
7	0xB	1	0x2F8 - 0x2FB	0x2FC - 0x2FF	M; H
7	0x0	1	0x038 - 0x03B	0x03C - 0x03F	H

1. 0x0C = 0000 1100 M

2. 0x2D0 = 0010 1101 0000 M

3. 0x04C = 1010 1100 M

4. 0x08 = 0000 1000 H

5. 0x2Fc = 0010 1111 1100 M

6. 0x160 = 0001 0110 0000 M

7. 0x02F8 = 0010 1111 1000 H

8. 0x0038 = 0011 1000 M

9. 0x2D4 = 0010 1101 0100 H

10. 0x04C = 1010 1100 H

11. 0x00B0 = 1011 0000

12. 0x160 = 0001 0110 0000 H

Ejercicio 5:

Un computador tiene una memoria principal de 64K bytes y una memoria caché de 1K bits para datos. La memoria caché utiliza correspondencia (mapeo) directa, con un tamaño de línea de 4 palabras de 16 bits c/u. Obtener:

- ¿Cuántos bits hay en los diferentes campos del formato de dirección de memoria principal?
- ¿Cuántas líneas contiene la memoria caché?
- ¿Cuántos bits hay en cada línea de la memoria caché y cómo se dividen según su función?

$$\text{Mem } \eta\eta\text{al} = 64 \text{ k bytes} = 2^6 \times 2^{10} \text{ Bytes} = 2^{16} = \text{direcciones de 16 bits}$$

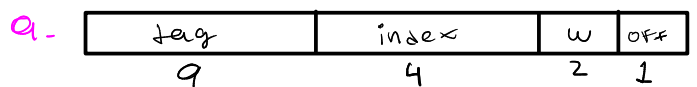
$$\text{Datos cache} = 1 \text{ k bits}$$

$$\text{Línea cache} = 4 \times 16 = 64 \text{ bits}$$

(bloque mem ppal)

$$\text{Cant. líneas} = \frac{1 \text{ k}}{64} = \frac{2^{10}}{2^6} = 2^4 = 16 \text{ líneas}$$

$$\text{Tags} = \frac{\text{bloques mem}}{\text{bloques cache}} = \frac{\text{cant w mem}}{\text{tamaño w q}} = \frac{2^{16}}{2^4} = 2^9 \Rightarrow \text{bits de tag} = 9$$



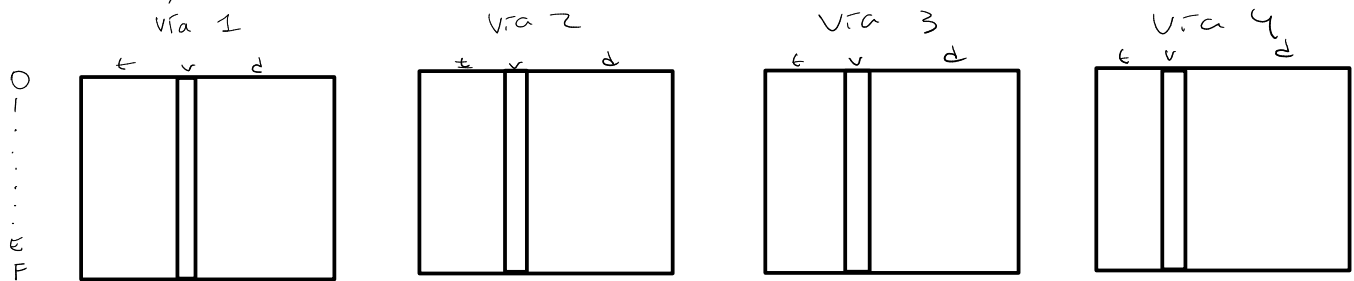
$$\begin{aligned} \text{c- Bits de una línea de caché} &= \text{bits de tag} + \text{bit de verificación} + \text{bits de datos} \\ &= 9 + 1 + 64 \end{aligned}$$

$$\text{Bits de una línea de caché} = 74$$

Ejercicio 6:

Una caché asociativa por conjuntos consta de 64 líneas, dividida en 4 vías. La memoria principal contiene 4K bloques de 128 palabras cada uno. Muestre el formato de dirección de memoria principal suponiendo que cada palabra es direccionable directamente en memoria.

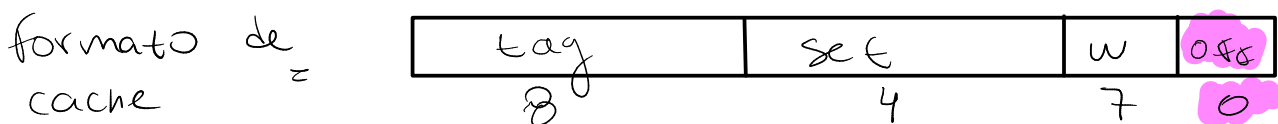
$$64 \text{ líneas} / 4 \text{ vías} = 16 \text{ líneas por vía (o sea 16 conjuntos / set)}$$



$$\begin{aligned} \text{Mem total} &= 4k \text{ bloques} \times 128 \text{ palabras c/u} \\ &= 2^2 \cdot 2^{10} \cdot 2^7 = 2^{19} \text{ palabras} \Rightarrow \text{direcciones de 19 bits} \end{aligned}$$

$$128 \text{ palabras} \times \text{bloque} \Rightarrow w = 7 \text{ bits}$$

$$\text{tags} = \frac{\text{Cant. bloques mem}}{\text{Cant. bloques cache}} = \frac{\frac{\text{total } w_{\text{mem}}}{w_{\text{mem en wq}}}}{2^4} = \frac{\frac{2^{19}}{2^7}}{2^4} = 2^8$$



Ejercicio 7:

Considere una CACHE con los siguientes parámetros:

- Criterio de correspondencia: Asociativa por conjuntos
- Asociatividad (N-vias): 2
- Tamaño de bloque: 2 words
- Tamaño de palabra (word): 32 bits
- Tamaño de la cache: 32K words (datos)
- Tamaño de dirección: 32 bits
- Cada palabra es directamente direccionable en memoria

Responder:

- Muestre el formato de dirección de memoria principal.
- ¿Cuál es el tamaño de toda el área de Tag de la cache, expresada en bits?
- Suponga que cada LINEA de la cache contiene además un bit de validación (V) y un tiempo de vida de 8 bits. Cual es el tamaño completo (expresado en bits) de un CONJUNTO de la cache, considerando datos, tags y los bits de status antes mencionados?

a.

tag	set	w	ok
18	13	1	0

$$\frac{32 \text{ K}}{\# \text{ Vías}} = 16 \text{ K c/vía} \quad \text{y} \quad \frac{16 \text{ K}}{2 \text{ words}} = 8 \text{ K líneas por vía}$$

$$\text{Tags} = \frac{\text{Total bloques mem}}{\text{Total bloques cache}} = \frac{\frac{\# W_{\text{mem}}}{\# W_{\text{mem}} / \text{bloque}}}{2^{13}} = \frac{2^{32}}{2^{13}} = 2^{19} \Rightarrow 19 \text{ bits el tag}$$

b. $2^3 \cdot 2^{10} \cdot 19 = 144 \text{ K bits} \cdot 2 = 288 \text{ K bits}$
(líneas de cache x vía) (bits de tag) (vías) (total área de tag)

c. Datos (por línea por vía): 64 bits (2 words)

Tag (por línea por vía): 19 bits

1 bit de verificación (por línea por vía)

8 bits de tiempo de vida (por línea por vía)

$$\text{Total bits (por línea por vía): } 64 + 19 + 1 + 8 = 92 \text{ b}$$

$$\text{Total bits de un conjunto} = 92 \cdot 2 = 184 \text{ b}$$

Ejercicio 8:

Sea un sistema con una memoria principal de 1M palabras divididas en 4K bloques, donde cada palabra es direccionable directamente en memoria. Definir el formato de la dirección de memoria principal en los siguientes casos, sabiendo que la memoria caché posee 64 líneas:

- Memoria caché con función de correspondencia directa.
- Memoria caché con función de correspondencia full-asociativa.
- Memoria caché con función de correspondencia asociativa de 8 vías.

$$\frac{1M \text{ palabras}}{4K \text{ bloques}} = \frac{2^{20}}{2^2 \cdot 2^{10}} = 2^8 = 256 \text{ palabras por bloque}$$

Líneas de caché: 64

$$\text{Tag: } \frac{\# \text{ bloques mem}}{\# \text{ líneas caché}} = \frac{4K}{2^6} = \frac{2^2 \cdot 2^{10}}{2^6} = 2^6$$

Mapeo directo:

tag	index	w	off
6	6	8	0

No hay "líneas de caché" $\Rightarrow$ $\# \text{ tags} = \# \text{ bloques de mem}$

Full-asociativa:

tag	w	off
12	8	0

$$\text{Líneas por vía: } 64 / 8 = 8 \Rightarrow w = 3^b$$

$$\Rightarrow \text{tags} = \frac{\# \text{ bloques de mem}}{8} = \frac{4K}{8} = \frac{2^{12}}{2^3} = 2^9 = 9 \text{ bits de seq}$$

Asociativa por conjuntos de 8 vías:

tag	Set	w	off
9	3	8	0

Ejercicio 9:

Considere que un sistema tiene una CACHE para DATOS de correspondencia ASOCIATIVA POR CONJUNTOS de 2 VÍAS de 256Kbyte y 4 palabras por línea, sobre un procesador de 32bits, CPI = 1, que resuelve todos los data y control hazard sin necesidad de stalls, tiene una memoria principal de 4G palabras de 1 byte cada una.

a. Muestre el formato de memoria principal.

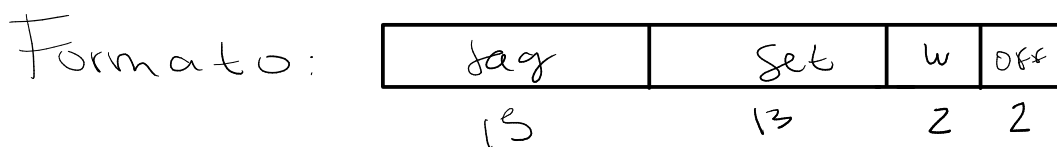
$$\text{Tamaño} \times \text{vía} : 256 \text{ KB} / 2 = 128 \text{ KB}$$

$$4 \text{ words} \times \text{línea} : 4 \times 32 \text{ b} = 128 \text{ b} \text{ y línea}$$

$$\Rightarrow \frac{2^7 \cdot 2^{10} \cdot 2^3}{2^7} = 8 \text{ K líneas (bits de set: 13)}$$

$$\text{Tags} : \frac{\# \text{bloques mem}}{\# \text{líneas cache}} = \frac{\frac{\# W_{\text{mem}}}{\# W_{\text{mem}} \times \text{byte}}}{2^{13}} = \frac{\frac{2^{32}}{2^4}}{2^{13}} = 2^{15} \text{ (bits de tag: 15)}$$

$$4 \text{ palabras por línea} \Rightarrow w : 2 \text{ b}$$



b. Considere ahora la ejecución en este sistema de los siguientes segmentos de código LEGv8 equivalentes para $N > 0$, donde $N \rightarrow X3$, start = 0x000203C y X6 contiene la dirección base del arreglo A[] del tipo uint32_t. Considerar que el resto de los registros están inicializados en 0 y la CACHE vacía al inicio de la ejecución de cada segmento.

Segmento #1:

```
start: ADD X9, X3, XZR
loop: CBZ X9, end
      LDURH X10, [X6, #0]
      LDURH X11, [X6, #8]
      ADD X12, X10, X11
      LSR X12, X12, #1
      ADDI X6, X6, #4
      SUBI X9, X9, #1
      B loop
end: ...
```

Segmento #2:

```
start: LDURH X10, [X6, #0]
      LDURH X11, [X6, #8]
      ADD X12, X10, X11
      LSR X12, X12, #1
      ADDI X6, X6, #4
      ADDI X9, X9, #1
      SUBS XZR, X9, X3
      B.NE start
end: ...
```

¿Cuántos MISS de CACHE se produjeron en 5 iteraciones del lazo para el segmento #1 si la dirección base del arreglo A es 0x00001008?

Segmento #1:
start: ADD X9, X3, XZR
loop: CBZ X9, end
LDURH X10, [X6, #0]
LDURH X11, [X6, #8]
ADD X12, X10, X11
LSR X12, X12, #1
ADDI X6, X6, #4
SUBI X9, X9, #1
B loop
end: ...

Nº iteración	Direcciones de acceso	Hit / Miss	Data que guarda
1	0x1008	Miss	[0x1008-0x100B] [0x100C-0x100F] [0x1010-0x1013] [0x1014-0x1017]
	0x1010	Hit	
2	0x100C	Hit	
	0x1014	Hit	
3	0x1010	Hit	
	0x1018	Miss	[0x1018-0x101B] [0x101C-0x101F] [0x1020-0x1023] [0x1024-0x1027]
4	0x1014	Hit	
	0x101C	Hit	
5	0x1018	Hit	
	0x1020	Hit	

- c. Ahora considere que se incorpora una CACHE exclusiva para INSTRUCCIONES de correspondencia FULL ASOCIATIVA de una sola línea de 8 palabras de 32 bits c/u. Considerando que cada MISS de caché supone una penalidad de 5 ciclos de clk. Determine qué segmento de código se ejecuta en menor tiempo para N = 4. Argumente su respuesta indicando los ciclos totales en cada caso. Suponer que el pipeline ya se encuentra en régimen y que la CACHE de DATOS solo produce aciertos en los accesos, por lo que no se tiene penalidades por acceso a datos.

$\Rightarrow$ Se guardan 8 instrucciones consecutivas

Segmento #2:

```
start: LDURH X10, [X6,#0]
```

```
LDURH X11, [X6,#8]
```

```
ADD X12, X10, X11
```

```
LSR X12, X12, #1
```

```
ADDI X6, X6, #4
```

```
ADDI X9, X9, #1
```

SUBS XZR, X9, X3

B.NE start

```
end: ...
```

```
end: ...
```

x9	instru	Hit / miss	clocks	Qué guardo en cache
0	a	Miss	1+5	a, b, c, d
	b	Hit	7	e, f, g, h
	c	Hit	8	
	d	Hit	9	
	e	Hit	10	
1	f	Hit	11	
	g	Hit	12	
	h	Hit	13	
2	a	Hit	14	
	e	Hit	18	
	f	Hit	19	
	h	Hit	21	
3	a	Hit	22	
	e	Hit	26	
	f	Hit	27	
	h	Hit	29	
4	a	Hit	30	
	e	Hit	34	
	f	Hit	35	
	h	Hit	37	
	i	Miss	43	i... i+7

Ejercicio 10:

Encuentre el AMAT (Average Memory Access Time) para un procesador con un tiempo de ciclo de 1 ns, un miss penalty de 20 ciclos de reloj, un miss rate de 5% fallos por instrucción y un tiempo de acceso a caché (incluida la detección de acierto) de 1 ciclo de reloj. Suponga que las penalizaciones por falla de lectura y escritura son las mismas e ignore otros stalls en la escritura.

$$\text{Ciclo de clk} = 1 \text{ ns}$$

$$\text{Miss rate} = 5\%$$

$$\text{Penalidad} = 20 \text{ ns (20 ciclos)} \quad \text{Access time} = 1 \text{ ns (Hit time)}$$

$$\begin{aligned} \text{AMAT} &= \text{Hit time} + \text{miss-rate} \cdot \text{penalty} \\ &= 1 \text{ ns} + 0,05 \cdot 20 \text{ ns} \end{aligned}$$

$$\text{AMAT} = 2 \text{ ns}$$

Ejercicio 11:

Una computadora posee un sistema de memoria jerárquica que consiste en dos cachés separadas (una de instrucciones y una de datos), seguidas por la memoria principal y un procesador ARM que trabaja a 1 GHz, con hit time = 1 ciclo de clock y CPI = 1.

- La caché de instrucciones es perfecta (siempre acierta), pero la caché de datos tiene un 15% de miss rate. En una falla de caché, el procesador se detiene durante 200 ns para acceder a la memoria principal y luego reanuda el funcionamiento normal. Teniendo en cuenta los errores de caché, ¿cuál es el tiempo promedio de acceso a la memoria?
- ¿Cuántos ciclos de reloj por instrucción (CPI) se requieren en promedio para instrucciones load y store, considerando las condiciones del punto a)?
- Asumiendo que la distribución de instrucciones se divide en las siguientes categorías: 25% loads, 10% stores, 13% branches y 52% instrucciones tipo R, ¿cuál es el CPI promedio?
- Suponiendo ahora que la caché de instrucciones tampoco es ideal y tiene un miss rate del 10%, ¿cuál es el CPI promedio considerando los fallos de ambas cachés?. Asuma la misma distribución de instrucciones de programa del punto c).

$$\text{Ciclo de clk} = 1 \text{ GHz} \Rightarrow \frac{1}{10^9} \text{ s} = 1 \text{ ns};$$

$$\text{Hit time} = 1 \text{ ns}; \text{ CPI} = 1$$

$$\text{a - Miss rate} = 15\%; \text{ Penalty} = 200 \text{ ns}$$

$$\text{AMAT} = 1 \text{ ns} + 0,15 \cdot 200 \text{ ns} = 31 \text{ ns}$$

$$\text{b - CPI/instru} = \text{CPI} + \left(\frac{\text{Memory Accesses}}{\text{Total Program Instructions}} \right) \cdot \text{miss. Penalty rate}$$

$$\text{CPI - IyS} = 1 \text{ ns} + 100/100 \cdot 0,15 \cdot 200 \text{ ns} = 31 \text{ ns} = 32 \text{ ciclos}$$

$$\text{c - CPI - prom} = 1 \text{ ns} + 0,35 \cdot 0,15 \cdot 200 \text{ ns} = 11,5 \text{ ns} = 11,5 \text{ ciclos}$$

$$\begin{aligned} \text{d - CPI - prom} &= 1 \text{ ns} + \underset{\text{(caché de instrus)}}{0,1 \cdot 200 \text{ ns}} + \underset{\text{(caché de datos)}}{0,35 \cdot 0,15 \cdot 200 \text{ ns}} \\ &= 31,5 \text{ ns} = 31,5 \text{ ciclos} \end{aligned}$$

Ejercicio 12:

La siguiente tabla muestra los datos de la caché de un solo nivel (L1) para dos procesadores distintos (P1 y P2). En ambos procesadores el tiempo de acceso a memoria principal es de 70ns y el 36% de las instrucciones acceden a la memoria de datos.

	L1 Size	L1 Miss Rate	L1 Hit Time
P1	2 KiB	8.0%	0.66ns
P2	4 KiB	6.0%	0.90ns

- Asumiendo que el *hit time* de la caché L1 determina el tiempo de ciclo de ambos procesadores, calcule sus respectivas frecuencias de CLK.
- ¿Cuál es el AMAT para ambos procesadores?
- Asumiendo un CPI de 1 (sin memory stalls) y considerando las penalidades derivadas de instrucciones y datos (mismo Miss Rate), ¿cuál es el CPI total para ambos procesadores? ¿Cuál de los dos procesadores es más rápido?

a- Hit time = ciclo de clock = 0,66 ns (P1) y 0,9 (P2)

$$Frecuencia_{P1} = \frac{1}{0,66 \times 10^{-9}} = 1,515 \text{ GHz}$$

$$Frecuencia_{P2} = \frac{1}{0,9 \times 10^{-9}} = 1,111 \text{ GHz}$$

b- $AMAT(P1) = 0,66 \text{ ns} + 70 \text{ ns} \cdot 0,08 = 6,26 \text{ ns}$

$$AMAT(P2) = 0,9 \text{ ns} + 70 \text{ ns} \cdot 0,06 = 5,1 \text{ ns}$$

c- $CPI = 1 (0,66 \text{ ns} / 0,9 \text{ ns})$

$$\begin{aligned} CPI_{P1} &= 0,66 \text{ ns} + 70 \cdot 0,08 + 70 \cdot 0,36 \cdot 0,08 \\ &= 8,276 \text{ ns} = \frac{8,276}{0,66} \text{ ciclos} = 12,54 \text{ ciclos} \end{aligned}$$

$$\begin{aligned} CPI_{P2} &= 0,9 + 70 \cdot 0,06 + 70 \cdot 0,36 \cdot 0,06 \\ &= 6,612 \text{ ns} = \frac{6,612}{0,9} \text{ ciclos} = 7,35 \text{ ciclos} \end{aligned}$$

El P2 es más rápido, ya que tiene menor AMAT y menor CPI promedio.